

Handling Imperfect Engineering Data

Engineering Computation with Python — Chapter 11

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

1 Errors, Exceptions and try/except

2 Common Exception Types

3 Writing Robust Code

4 Floating-Point Pitfalls

5 Missing Data

6 Outliers

1 **Errors, Exceptions and try/except**

2 **Common Exception Types**

3 **Writing Robust Code**

4 **Floating-Point Pitfalls**

5 **Missing Data**

6 **Outliers**

Two Different Things Go Wrong

This chapter deals with both.

An error in your **code**

A file that does not exist, a division by zero, a value of the wrong type. Python detects these **immediately** and stops the program with an exception — unless you handle it.

A flaw in your **data**

A sensor dropout, a power-spike glitch, a physically implausible value. Python does **not** detect these — a corrupted reading is still a perfectly valid floating-point number.

The job split

A robust program **survives** the first kind of problem. A careful engineer **investigates and repairs** the second.

In Python, errors fall into two broad categories:

Syntax errors

Caught by Python **before** the program even runs. Code violates the grammatical rules of the language.

Exceptions

Occur **while** the program is running. Something unexpected happens during execution, even though the code is syntactically correct.

Key fact

Exceptions are a **normal** part of real programs — not rare bugs. Any program that interacts with users, files, or external systems should expect them and be designed to handle them.

The basic structure for handling exceptions has four parts:

```
try:
    # risky code
except ErrorType:
    # handle the error
else:
    # runs if no error
finally:
    # always runs (cleanup)
```

- **try** – code that might raise an exception.
- **except** – catches and handles a specific exception type, if one occurs.
- **else** – runs only if **try** completed with no exception.
- **finally** – always runs, exception or not; the right place for cleanup.

An exception is a runtime error object created when Python cannot continue executing a piece of code normally.

Not handled

The program **crashes**.

Handled

The program **recovers and continues**.

Typical situations that raise exceptions:

- Dividing by zero.
- Converting text to a number when the text is not a valid number.
- Opening a file that does not exist.

1 Errors, Exceptions and try/except

2 **Common Exception Types**

3 Writing Robust Code

4 Floating-Point Pitfalls

5 Missing Data

6 Outliers

Common Exception Types: The Full Roster

Python defines many built-in exception types. Each one tells you specifically what went wrong.

Exception	Meaning
ValueError	Correct type, invalid or inappropriate value
TypeError	Operation on an incompatible/unsupported type
ZeroDivisionError	Division (or modulo) by zero
IndexError	Sequence index out of range
KeyError	Dictionary key does not exist
FileNotFoundError	File or path could not be found
ImportError	Module or name cannot be found or loaded
AttributeError	Object has no such attribute/method
NameError	Name has not been defined
SyntaxError	Code violates Python's grammar rules
IndentationError	Incorrect or missing indentation
RuntimeError	Generic runtime failure (e.g. infinite recursion)

The next few slides look closely at the five you will meet most often. The rest appear in the reference table at the end of this section.

Correct **type**, but an invalid or inappropriate **value**.

```
x = int("abc")  
print(x)
```

```
ValueError: invalid literal for int()
```

Common causes:

- Converting invalid values
- Passing an invalid value to a function
- Incorrect input format
- Values outside the expected range

How to fix

Ensure the value matches the expected format; validate user input before conversion; use `try/except`.

TypeError and ZeroDivisionError

TypeError — incompatible data types.

```
x = 10  
y = "5"  
print(x + y)
```

TypeError: unsupported operand type(s)

ZeroDivisionError — division by zero.

```
x = 10  
y = 0  
print(x / y)
```

ZeroDivisionError: division by zero

How to fix

Convert types properly (`int()`, `str()`, `float()`); check a value is nonzero before dividing; use `type()` to debug.

IndexError and KeyError

IndexError — sequence index out of range.

```
nums = [1, 2, 3]  
print(nums[5])
```

IndexError: list index out of range

KeyError — dictionary key does not exist.

```
data = {"name": "Amanda"}  
print(data["email"])
```

KeyError: 'email'

How to fix

Check length with `len()` before indexing; check key existence with `in`, or use `.get()` to avoid the error entirely.

Python cannot find or access the requested file.

```
file = open("data.txt", "r")  
content = file.read()  
file.close()
```

FileNotFoundError: No such file or directory

Common causes:

- File does not exist, or wrong name/extension.
- Incorrect file path, or file in a different directory.

How to fix

Check the file name and extension; verify the path; confirm the file exists; use an absolute path if needed.

Six more exception types appear in the book, each with the same pattern of common causes and fix. Quick summary:

- **ImportError** — a module or name cannot be found/loaded (e.g. a typo: `from math import sqr`).
- **AttributeError** — an object has no such attribute/method (e.g. `(10).append(5)`).
- **NameError** — a variable or function was never defined, or is misspelled.
- **SyntaxError** — caught before the program even runs; a missing colon, bracket, or quote.
- **IndentationError** — incorrect or missing spacing; Python uses indentation to define structure.
- **RuntimeError** — a generic failure during execution, e.g. infinite recursion.

A related but distinct problem

A **logical error** raises no exception at all — the code runs, but produces the **wrong** output. These are often the hardest bugs to find, because Python gives no error message.

1 Errors, Exceptions and try/except

2 Common Exception Types

3 **Writing Robust Code**

4 Floating-Point Pitfalls

5 Missing Data

6 Outliers

A very common source of exceptions is user input:

```
x = int(input("Enter a number"))
```

This line can fail in more than one way:

- The user types text instead of a number → `ValueError`.
- The user presses Enter without typing anything → `ValueError`.

Multiple except clauses

Different problems need different handlers — several except clauses can attach to one `try`, so the program responds to each kind of error without crashing.

The underlying lesson: programs must not blindly trust their users.

```
try:
    x = int(input("Enter a number: "))
    print(10 / x)
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Files are external resources too

They may not exist, paths may be wrong, permissions may be denied. Exception handling around file operations protects against corrupted or locked files and prevents data loss.

The file may fail to open, or fail partway through reading. The `finally` block runs regardless — the right place to close a file.

```
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found!")
except IOError:
    print("Error reading file!")
finally:
    print("Closing file...")
    try:
        f.close()
    except:
        pass
```

The nested `try` inside `finally` guards against the case where `f` was never opened in the first place.

1 Errors, Exceptions and try/except

2 Common Exception Types

3 Writing Robust Code

4 **Floating-Point Pitfalls**

5 Missing Data

6 Outliers

Some errors are silent — no exception is raised, yet the result is wrong.

```
a = 0.1 + 0.2
if a == 0.3:
    print("a is 0.3")
```

This print statement never executes

0.1 + 0.2 does not exactly equal 0.3 in floating-point arithmetic.

Why: floats are stored in binary, not decimal; many decimal numbers cannot be represented exactly; == checks for an **exact** match.

Compare with a **tolerance** rather than exact equality:

$$|a - b| < \varepsilon \implies \text{treat } a \text{ and } b \text{ as equal}$$

```
epsilon = 1e-9  # acceptable error margin
a = 0.1 + 0.2
b = 0.3
if abs(a - b) < epsilon:
    print("Equal within epsilon")
```

This works, but it is easy to misuse — e.g. choosing an epsilon inappropriate for the magnitude of the numbers compared.

Python's standard library handles this automatically — safer and clearer than writing manual checks.

```
import math
print(math.isclose(0.1 + 0.2, 0.3))           # True
print(math.isclose(1.0001, 1.0))              # False
print(math.isclose(1.0001, 1.0, rel_tol=1e-3)) # True
```

- `rel_tol` — relative tolerance (a fraction of the larger value).
- `abs_tol` — absolute tolerance, useful near zero.

- Errors are a normal part of real programs.
- `try/except/finally` prevents crashes and protects resources such as open files.
- Catching a **specific** exception type is far better practice than a bare `except :`, which hides faults you did not anticipate.
- Floating-point numbers are inexact — avoid `==`; use `math.isclose()`.

“Good programs don’t avoid errors — they handle them.”

1 Errors, Exceptions and try/except

2 Common Exception Types

3 Writing Robust Code

4 Floating-Point Pitfalls

5 **Missing Data**

6 Outliers

A missing value in a NumPy array of floats is represented by **NaN** (Not-a-Number), an IEEE 754 special value.

```
import numpy as np

x = np.array([1.0, 2.0, np.nan, 4.0])

print(x + 1)           # [2.  3. nan  5.] -- NaN spreads
print(x == x)          # [True True False True] -- NaN != NaN
print(np.mean(x))      # nan -- one bad value poisons the result
```

The practical danger

A single dropped sensor reading, left unhandled, silently turns an entire statistical summary into nan.

Should This Gap Even Be Filled?

Before reaching for `np.interp()`, pause: the right answer depends on what the gap **represents**.

- **Short gap, smooth signal?** Usually safe to **interpolate**.
- **Isolated entry, not a time series?** No "neighbours in time" — a **mean, median, or model-based estimate** may fit better.
- **Large fraction missing?** Filling it manufactures more signal than was measured — **discard the segment** or investigate the sensor instead.
- **Is "missing" itself meaningful?** Then **keep it as missing and document why**.

Notice

Three of these four branches do **not** end in interpolation. Which tool to reach for matters more than knowing any one tool well.

Before a gap can be repaired, it must first be **located**. `np.isnan()` returns a Boolean array, True wherever a value is NaN:

```
raw_data = np.array([12.5, np.nan, 13.1, 12.9, np.nan, 13.4])

is_missing = np.isnan(raw_data)
print(is_missing)  # [False True False False True False]

clean_data = raw_data[~np.isnan(raw_data)]
print(clean_data)  # [12.5 13.1 12.9 13.4]
```

The `~` operator flips each Boolean, turning "is missing" into "is not missing". Filtering shortens the array and discards the **position** of the missing readings — for a time series, that position is often exactly what you need.

Repairing a Signal by Interpolation

`np.interp()` estimates each missing value from its neighbours, preserving the full length and timing of the signal:

```
np.interp(x[nans], x[~nans], y[~nans])
```

- `x[nans]` — positions where a value is missing.
- `x[~nans]` — positions with a good value.
- `y[~nans]` — the good values themselves.

EE Focus: Patching a Sensor Dropout

A temperature sensor logging twice a second suffers a 3-second dropout (loose connector). Seven consecutive readings are lost.

Worked Example: Sensor Dropout Repair

```
t = np.arange(0, 20, 0.5)
temp_true = 25 + 3 * np.sin(0.3 * t)

temp_measured = temp_true.copy()
dropout = (t >= 8) & (t <= 11)
temp_measured[dropout] = np.nan

nans = np.isnan(temp_measured)
temp_repaired = temp_measured.copy()
temp_repaired[nans] = np.interp(t[nans], t[~nans],
                                temp_measured[~nans])
```

Number of missing samples: 7 Max repair error: 0.165 deg C

But not a universal rule

Interpolation assumes the signal changes **smoothly** across the gap. If a step change happened inside the missing interval, a straight line can manufacture a plausible but false trajectory.

1 Errors, Exceptions and try/except

2 Common Exception Types

3 Writing Robust Code

4 Floating-Point Pitfalls

5 Missing Data

6 **Outliers**

A missing value is easy to define: the sensor recorded nothing. An outlier is trickier — the sensor **did** record something; the question is whether to trust it.

A statistical description, not a verdict

An outlier is a measurement unusually far from the surrounding data. That alone does not tell you whether it is a fault or the most important sample in the dataset.

Distinguishing fault from genuine event is far more important — and far harder — than choosing a detection formula.

Type A — Obvious measurement error

A temperature trace near 26.5°C shows one sample at 91.3°C , then returns to 26.7°C . Nothing heats and cools that fast — almost certainly a glitch.

Type B — Suspicious but uncertain

One sample at 31.3°C . Could be a small glitch, or the leading edge of a real transient. **Do not correct automatically.**

Type C — Genuine event

$26 \rightarrow 27 \rightarrow 28 \rightarrow 45 \rightarrow 43 \rightarrow 35 \rightarrow 29^{\circ}\text{C}$ — rises and falls smoothly. Very likely the most important measurement in the dataset.

A statistical test (Z-score, IQR — Chapter ??) flags Type A, B, and C readings **equally well**, because statistically they all look the same: unusually far from the rest of the data.

Only engineering judgement tells them apart

Looking at neighbouring samples, the physical system, and other measured channels is what distinguishes a fault from a real event — a formula alone cannot.

Detect → Flag → Investigate → Decide → Document

Worked Example: Investigating a Suspicious Reading

The bench temperature log shows one sample far above its neighbours:

```
t = np.arange(0, 30, 1.0)
temp = 25 + 1.5*np.sin(0.2*t) + np.random.normal(0, 0.1, t.size)
temp[18] = 45.8 # a single sample looks far too high
```

The investigation, step by step:

1. **Plot it** — the point clearly stands apart.
2. **Compare with neighbours** — no gradual rise or fall leading up to it.
3. **Check physical plausibility** — is a 20 °C jump in one second consistent with the board's thermal mass? Usually not.
4. **Check other channels** — are voltage and current normal at sample 18?

Only Then: Flag and Repair

Only once the reasoning points to a fault does it make sense to flag and repair – using the same tool already used for missing data:

```
is_suspect = np.zeros(t.size, dtype=bool)
is_suspect[18] = True

temp_flagged = temp.copy()
temp_flagged[is_suspect] = np.nan

temp_repaired = temp_flagged.copy()
temp_repaired[is_suspect] = np.interp(
    t[is_suspect], t[~is_suspect], temp_flagged[~is_suspect])
```

The conclusion could reverse

If voltage and current also rose at sample 18, this is a genuine Type C event – interpolating it away would erase the most important measurement in the log.

Imputation — replacing a missing value with an estimate. Interpolation is one way; it is not the only way.

Mean / median

Fast and simple. Good when missing values are scattered, not contiguous. A median resists outliers better than a mean.

Forward-fill

Repeats the last known good value. Suits signals that change only occasionally (a status flag, a calibration value).

Interpolation

Estimates from neighbours. Suits a smoothly varying signal with a short gap.

There is no universally "correct" choice — it depends on how the underlying quantity actually behaves between measurements.

Problem	Example	First question	Possible action
Missing	No ADC reading	Why is it missing?	Leave, remove, or interpolate
Noise	Small random variation	Within expected uncertainty?	Usually retain
Outlier	One extreme reading	Is it physically valid?	Retain/flag/remove – after investigating
Sensor failure	Impossible value	Is the system faulty?	Remove or mark invalid
Genuine event	Sudden but real change	Physically meaningful?	Keep
Long gap	Sensor offline	Can interpolation be trusted?	Usually do not interpolate blindly

- Errors in **code** are detected by Python; flaws in **data** are not, because a corrupted reading is still a valid number.
- Catching a **specific** exception beats a bare `except:`, which hides faults you did not anticipate.
- Never compare floats with `==` — use `math.isclose()`.
- NaN is contagious: one bad value poisons a mean. `np.isnan()` locates it; NaN-aware functions skip it — skipping is not the same as repairing.
- Deciding **what** to do about a gap matters more than knowing the functions.
- An outlier is a statistical description, not a verdict: **Detect** → **Flag** → **Investigate** → **Decide** → **Document**.

Next: Interpolation, Curve Fitting and Calibration